



Spark Getting Started

University of Virginia

DS 7200: Distributed Computing

Last Updated: August 20, 2026

SOURCES

Learning Spark, First Edition

Chapter 1: Introduction to Data Analysis with Spark

Chapter 2: Getting Started

OBJECTIVES

- Spark background
- Setup and installation
- Basic concepts
- Minimal code examples
- Running Spark: Interactive Session
- Running Spark: Command Line

CONCEPTS

- Cluster: a set of connected computers (nodes)
- Functional programming

- SparkSession - single point of entry to interact w Spark functionality
 - Resilient Distributed Datasets (RDDs) - Spark's fundamental abstraction for distributed data and computation
 - Dataset
 - Driver Program - contains application main function, defines RDDs on cluster, applies operations to them.
 - Worker Node or Executor - the units that perform tasks
-

Spark Benefits

- Designed to be fast
no waiting around for hours, need to work interactively with data
 - Designed to handle big data
 - General Purpose
Unlike Hadoop, several modules in one place:
 - Machine learning
 - SQL queries
 - Streaming
 - Graph analytics
 - Caching is possible, so intermediate data can be stored in memory on workers
 - Highly accessible: simple APIs to Python, Java, Scala, R, SQL
Integrates w other Big Data tools such as Hadoop, Cassandra
Can access HDFS data, Amazon S3, and others
-

Set up a Spark Session with Minimal Parameters

- use local machine as master
- name the app

```
In [27]: from pyspark.sql import SparkSession

spark = SparkSession.builder \
```

```
.master("local") \
.appName("pyspark_test") \
.getOrCreate()
```

```
In [28]: # print info about the session
spark
```

Out[28]: **SparkSession - in-memory**

SparkContext

Spark UI

Version	v4.2.0
Master	local
AppName	pyspark_test

```
In [29]: # need this for working with RDDs
sc = spark.sparkContext
```

RDDs (Resilient Distributed Datasets)

RDD objects are the most basic abstraction in Spark. They have these properties:

- **resilient:** list of dependencies instructs Spark how the RDD is constructed from inputs.
In the event the RDD is compromised, Spark can recreate it from dependencies.
- uses `partitions` for storing pieces of the data. Spark automatically partitions RDDs and distributes the partitions across nodes in the cluster.
- **distributed:** placing the partitions across nodes in the cluster allows for storing massive datasets that wouldn't fit on a single machine.

RDD History

Before Spark 2.0, the main programming interface of Spark was the *Resilient Distributed Dataset (RDD)*.

Starting with Spark 2.0, the *Dataset* and *DataFrame* objects were released. They are built up from RDDs.

We work with DataFrames later.

The RDD interface is still supported

Computing

We will look at several examples.

Example 1: Read lines from text file

```
In [30]: data_filename = 'README.txt'
```

```
In [31]: lines = sc.textFile(data_filename)
```

```
In [32]: lines.count()
```

```
Out[32]: 41
```

```
In [33]: lines.first()
```

```
Out[33]: 'Apache Spark'
```

```
In [34]: lines.collect()
```

```
Out[34]: ['Apache Spark',  
          'Spark is a unified analytics engine for large-scale data processing.  
          It provides high-level APIs in Scala, Java, Python, and R, and an  
          optimized engine that supports general computation graphs for data an  
          alysis. It also supports a rich set of higher-level tools including S  
          park SQL for SQL and DataFrames, MLlib for machine learning, GraphX f  
          or graph processing, and Structured Streaming for stream processin  
          g.',  
          '',  
          'https://spark.apache.org/',  
          '',  
          'Jenkins Build AppVeyor Build PySpark Coverage',  
          '',  
          'Online Documentation',  
          'You can find the latest Spark documentation, including a programmin  
          g guide, on the project web page. This README file only contains basi  
          c setup instructions.',  
          '',  
          'Building Spark',  
          'Spark is built using Apache Maven. To build Spark and its example p  
          rograms, run:',  
          '',  
          './build/mvn -DskipTests clean package',  
          '(You do not need to do this if you downloaded a pre-built packag  
          e.)',  
          '',  
          'You can build Spark using more than one thread by using the -T opti  
          on with Maven, see "Parallel builds in Maven 3". More detailed docume  
          ntation is available from the project site, at "Building Spark".',
```

```

'',
'For general development tips, including info on developing Spark us
ing an IDE, see "Useful Developer Tools".',
'',
'Interactive Scala Shell',
'The easiest way to start using Spark is through the Scala shell:',
'',
'./bin/spark-shell',
'Try the following command, which should return 1,000,000,000:',
'',
'scala> spark.range(1000 * 1000 * 1000).count()',
'Interactive Python Shell',
'Alternatively, if you prefer Python, you can use the Python shel
l:',
'',
'./bin/pyspark',
'And run the following command, which should also return 1,000,000,0
00:',
'',
'>>> spark.range(1000 * 1000 * 1000).count()',
'Example Programs',
'Spark also comes with several sample programs in the examples direc
tory. To run one of them, use ./bin/run-example <class> [params]. For
example:',
'',
'./bin/run-example SparkPi',
'will run the Pi example locally.',
'',
'You can set the MASTER environment variable when running examples t
o submit examples to a cluster. This can be a mesos:// or spark:// UR
L, "yarn" to run on YARN, and "local" to run locally with one thread,
or "local[N]" to run locally with N threads. You can also use an abbr
eviated class name if the class is in the examples package. For insta
nce:']

```

```
In [35]: type(lines.collect())
```

```
Out[35]: list
```

```
In [36]: lines.collect()[1]
```

```
Out[36]: 'Spark is a unified analytics engine for large-scale data processing.
It provides high-level APIs in Scala, Java, Python, and R, and an opt
imized engine that supports general computation graphs for data analy
sis. It also supports a rich set of higher-level tools including Spar
k SQL for SQL and DataFrames, MLlib for machine learning, GraphX for
graph processing, and Structured Streaming for stream processing.'
```

```
In [37]: type(lines.collect()[0])
```

```
Out[37]: str
```

Example 2: Text Search - apply filter and print all lines containing "Spark"

```
In [38]: spark_lines = lines.filter(lambda x: "Spark" in x)
```

```
In [39]: # return list of first 5 records
spark_lines.take(5)
```

```
Out[39]: ['Apache Spark',
          'Spark is a unified analytics engine for large-scale data processing. It provides high-level APIs in Scala, Java, Python, and R, and an optimized engine that supports general computation graphs for data analysis. It also supports a rich set of higher-level tools including Spark SQL for SQL and DataFrames, MLlib for machine learning, GraphX for graph processing, and Structured Streaming for stream processing.',
          'Jenkins Build AppVeyor Build PySpark Coverage',
          'You can find the latest Spark documentation, including a programming guide, on the project web page. This README file only contains basic setup instructions.',
          'Building Spark']
```

```
In [40]: type(spark_lines)
```

```
Out[40]: pyspark.core.rdd.PipelinedRDD
```

Example 3: Word Count

```
In [41]: # Read the file into an RDD
lines = sc.textFile(data_filename)
```

```
In [42]: type(lines)
```

```
Out[42]: pyspark.core.rdd.RDD
```

```
In [43]: words = lines.flatMap(lambda x: x.split())
```

```
In [44]: words.take(5)
```

```
Out[44]: ['Apache', 'Spark', 'Spark', 'is', 'a']
```

```
In [45]: wordcounts = words.map(lambda x: (x, 1)) \
          .reduceByKey(lambda x,y:x+y) \
          .map(lambda x:(x[1],x[0])) \
          .sortByKey(False)
```

```
In [46]: wordcounts.take(10)
```

```
Out[46]: [(13, 'the'),
          (11, 'Spark'),
          (7, 'to'),
          (6, 'a'),
          (6, 'for'),
          (6, 'and'),
          (6, 'can'),
          (6, 'run'),
          (5, 'is'),
          (5, 'using')]
```

TRY FOR YOURSELF (UNGRADED EXERCISES)

1. Convert the Word Count example into a function called `word_count()`.

The function assumes you have already read in the text file into `lines`.

It should take two inputs:

- `lines` this is the RDD containing text
- `num_records` this is the number of wordcount pairs to return

It should output a list of the `num_records` most frequent word count pairs.

Enter the code for `word_count()` in the cell below.

```
In [47]: ## definition of word_count()
def word_count(lines, num_records):
    words = lines.flatMap(lambda x: x.split())
    wordcounts = words.map(lambda x: (x, 1)) \
                    .reduceByKey(lambda x,y:x+y) \
                    .map(lambda x:(x[1],x[0])) \
                    .sortByKey(False)
    return wordcounts.take(num_records)
```

Now test that `word_count()` returns the expected result.

Also insure that the output type is a list.

```
In [49]: ## test function: word_count()
print(word_count(lines, 10))
## calling type(output) should return a list
print(type(word_count(lines, 10)))
```

```
[(13, 'the'), (11, 'Spark'), (7, 'to'), (6, 'a'), (6, 'for'), (6, 'an
d'), (6, 'can'), (6, 'run'), (5, 'is'), (5, 'using')]
<class 'list'>
```

SOLUTIONS

```
In [23]: def word_count(lines, num_records):
```

```
wordcounts = lines.flatMap(lambda x: x.split()) \
    .map(lambda x: (x, 1)) \
    .reduceByKey(lambda x,y:x+y) \
    .map(lambda x: (x[1],x[0])) \
    .sortByKey(False)

return(wordcounts.take(num_records))
```

```
In [24]: out = word_count(lines, 10)
out
```

```
Out[24]: [(13, 'the'),
          (11, 'Spark'),
          (7, 'to'),
          (6, 'a'),
          (6, 'for'),
          (6, 'and'),
          (6, 'can'),
          (6, 'run'),
          (5, 'is'),
          (5, 'using')]
```

```
In [25]: type(out)
```

```
Out[25]: list
```